PANCHSHEEL PUBLIC SCHOOL

COMPUTER SCIENCE PROJECT

A PROJECT ON

COMPUTER SALES SERVICE SYSTEM

SUBMITTED BY:

- **Name: *Anubhav Arya***
- **Class: XII 'A'**
- **Roll.No.:**
- **Under the guidance of: *Ms. Sangeeta Yadav***

CONTENTS

1. Certificate
2. Acknowledgement
3. Introduction
4. About the programming languages (Python & SQL)
5. Libraries
6. Modules
7. Functions used in project
8. Source code with Breakdowns
9. Output (Functioning)
10. Bibliography

CERTIFICATE

This is to certify that the project on Courier Service System reported here has been successfully completed and submitted by *Anubhav Arya* of class XII 'A' under the guidance of *Ms. Sangeeta Yadav* for the academic session 2024-2025 in partial fulfilment of practical examination to be conducted by Central Board of Secondary Education [C.B.S.E.]

Subject Teacher's Signature

External Teacher's Signature

ACKNOWLEDGEMENT

It gives me immense pleasure to present this project. It would not have been possible without the kind support of my Computer Science Teacher *Ms. Sangeeta Yadav* under whose guidance and constant supervision, the project was finally brought to the present state. I would like to express my gratitude towards my parents for their kind cooperation and encouragement which helped me in completion of this project. I thank them for providing me with ample resources for the project.

I am also thankful to our respected principal ma'am *Ms. Bharti Mehta* and the whole Panchsheel Public School for giving me such an amazing opportunity to prepare this project. I would also like to thank my friends for providing constant support and necessary weblinks for the completion of this project.

INTRODUCTION

In the modern world, technology has become an integral part of our daily lives. One of the most significant advancements is the evolution of online services, which has revolutionized the way we conduct business. Among various online services, the Online Computer Sales Service System stands out as a remarkable innovation. This system leverages the power of the internet to facilitate the buying and selling of computers and related accessories, providing an efficient, convenient, and user-friendly platform for both sellers and customers.

The Need for Online Computer Sales Service System

The advent of the internet and digital technologies has transformed traditional business models, leading to the rise of e-commerce. The need for an Online Computer Sales Service System arises from several factors:

1. **Convenience**: Unlike traditional offline stores, an online platform allows customers to browse, compare, and purchase computers from the comfort of their homes at any time of the day.

2. **Wide Range of Choices**: Online stores can offer a much larger inventory than physical stores, giving customers access to a wide variety of brands, models, and specifications.

3. **Cost-Effectiveness**: Operating an online store often incurs lower overhead costs compared to maintaining a physical

store, which can lead to more competitive pricing for customers.

4. **Information Accessibility**: Customers can easily access detailed information, reviews, and ratings about products, enabling them to make informed purchasing decisions.
5. **Global Reach**: Online platforms break geographical barriers, allowing businesses to reach a global audience and customers to access products from around the world.

Advantages Over Offline Stores

The Online Computer Sales Service System offers several advantages over traditional offline stores:

1. **24/7 Availability**: Online stores are accessible at all times, eliminating the constraints of store hours and providing flexibility to customers.
2. **User-Friendly Experience**: Modern online platforms are designed to be intuitive and user-friendly, often featuring advanced search options, filters, and personalized recommendations.
3. **Reduced Physical Infrastructure**: Online stores do not require physical space for showcasing products, which reduces the need for expensive retail space and related costs.
4. **Efficient Inventory Management**: Online systems can integrate with inventory management software, ensuring real-time updates and better control over stock levels.
5. **Customer Reviews and Feedback**: Online platforms enable customers to leave reviews and ratings, helping

others make better choices and providing valuable feedback to sellers.

Limitations of Offline Stores Leading to Online Transition

While offline stores have their own charm and benefits, several limitations have led to the emergence and popularity of online sales systems:

1. **Limited Operating Hours**: Offline stores have fixed operating hours, which may not be convenient for all customers.
2. **Geographical Constraints**: Customers are limited to stores within their vicinity, which can restrict access to a broader range of products.
3. **Higher Operational Costs**: Physical stores incur significant costs for rent, utilities, and staffing, often leading to higher prices for customers.
4. **Limited Product Information**: In a physical store, obtaining detailed product information can be time-consuming, and the availability of knowledgeable staff may vary.
5. **Inconvenience of Travel**: Customers need to travel to the store, which can be time-consuming and inconvenient, especially for those with busy schedules.

Conclusion

The Online Computer Sales Service System represents a significant advancement in the way we buy and sell computers and related accessories. By addressing the limitations of traditional offline stores, this system offers numerous benefits such as convenience, cost-effectiveness, a wider range of choices, and accessibility. As technology continues to evolve, the shift towards online services is likely to grow, making e-commerce platforms an essential part of our daily lives. This transition not only enhances the shopping experience for customers but also provides businesses with opportunities to expand their reach and optimize their operations in an efficient and effective manner.

ABOUT THE PROGRAMMING LANGUAGES

1. PYTHON

Python is a high-level, general-purpose programming language. Its design philosophy emphasizes code readability with the use of significant indentation.

Python is dynamically typed and garbage-collected. It supports multiple programming paradigms, including structured (particularly procedural), object-oriented and functional programming.

Guido van Rossum began working on Python in the late 1980s as a successor to the ABC programming language and first released it in 1991 as Python 0.9.0. Python 2.0 was released in 2000. Python 3.0, released in 2008, was a major revision not completely backward-compatible with earlier versions. Python 2.7.18, released in 2020, was the last release of Python 2.

Python consistently ranks as one of the most popular programming languages, and has gained widespread use in the machine learning community.

Its core philosophy is summarized in the Zen of Python (PEP 20), which includes aphorisms such as:

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Readability counts.

GUIDO VAN ROSSUM

ADVANTAGES OF PYTHON:

1. Easy to read, learn and code

Python is a high-level language and its syntax is very simple. It does not need any semicolons or braces and looks like English. Thus, it is beginner-friendly. Due to its simplicity, its maintenance cost is less.

2. Dynamic Typing

In Python, there is no need for the declaration of variables. The data type of the variable gets assigned automatically during runtime, facilitating dynamic coding.

3. Free, Open Source

It is free and also has an open-source licence. This means the source code is available to the public for free and one can do modifications to the original code. This is a very useful feature that helps companies or people to modify according to their needs and use their version.

4. Portable

Python is also platform-independent. If we write the code on one of the Windows, Mac, or Linux operating systems, then we can run the same code on the other OS with no need for any changes. This is called Write Once Run Anywhere (WORA).

5. Extensive Third-Party Libraries

Python comes with a wide range of libraries like NumPy, Pandas, Tkinter, Django, etc. The python package installer (PIP) helps you install these libraries in your interpreter/ IDLE. These libraries have different modules/ packages. Using these make the coding process easier and makes it look simple.

6. Wide Range of Applications

Python has many applications like web development, making desktop GUIs, app development, artificial intelligence, data science, etc. It has become a preferred language by the professionals in many areas like engineering, mathematics and science.

LIMITATIONS OF PYTHON:

1. It's Simple Nature

Its simplicity is making it hard for a programmer to adjust to the syntax of the other programming languages. For example, a coder might forget to declare a variable in C falling into an error.

2. Slow Speed and Memory Inefficient

The interpreter in Python executed the code line by line, which increases the overall time. The dynamic typing feature also decreases the speed. This is because it has to do extra work during runtime.

3. Weak Mobile Computation

Python has many applications on the server-side. But it is hardly seen on the client-side or in mobile applications. The main reasons for this are:

- It occupies a lot of memory.
- This makes the process slow.
- It also does not facilitate security.

4. Poor Database Access

Python databases are weak compared to other technologies like JDBC (Java DataBase Connectivity) and ODBC (Open DataBase Connectivity).

5. Runtime Errors due to dynamic typing

Because it's a dynamically typed language, we do not need to declare any variable and a variable storing an integer can store a string later. This leads to runtime errors while doing operations.

2. STRUCTURED QUERY LANGUAGE (SQL)

Structured Query Language (SQL) is a domain-specific language used to manage data, especially in a relational database management system (RDBMS). It is particularly useful in handling structured data, i.e., data incorporating relations among entities and variables.

When data needs to be retrieved from a database, SQL is used to make the request. The DBMS processes the SQL query retrieves the requested data and returns it to us.

It allows users to store, manipulate, and retrieve data efficiently in databases like MySQL, SQL Server, Oracle, and more.

SQL RULES

The rules for writing SQL queries are:

- A ';' is used to end SQL statements.
- Identifiers, operator names, and literals are separated by one or more spaces or other delimiters.
- A comma (,) separates parameters without a clause.
- A space separates a clause.
- Reserved words cannot be used as identifiers unless enclosed with double quotes.
- Identifiers can contain up to 30 characters.
- Identifiers must start with an alphabetic character.
- Characters and date literals must be enclosed within single quotes.
- Numeric literals can be represented by simple values.

ADVANTAGES OF SQL

SQL has many advantages which makes it popular and highly demanded. Some of its advantages are:

1. Faster Query Processing

Large amount of data is retrieved quickly and efficiently. Operations like Insertion, deletion, manipulation of data is also done in almost no time.

2. Standardized Language

Due to documentation and long establishment over years, it provides a uniform platform worldwide to all its users.

3. Portable

It can be used in programs in PCs, server, laptops independent of any platform (Operating System, etc). Also, it can be embedded with other applications as per need/requirement/use.

4. Interactive Language

Easy to learn and understand, answers to complex queries can be received in seconds.

5. Security

SQL databases have built-in security features that help protect data from unauthorized access, such as user authentication, encryption, and access control.

DISADVANTAGES OF SQL

1. Complex Interface

SQL has a difficult interface that makes few users uncomfortable while dealing with the database.

2. Cost

Some versions are costly and hence, programmers cannot access it.

3. Partial Control

Due to hidden business rules, complete control is not given to the database.

4. Limited Flexibility

SQL databases are less flexible than NoSQL databases when it comes to handling unstructured or semi-structured data, as they require data to be structured into tables and columns.

5. Complexity

SQL databases can be complex to set up and manage, requiring skilled database administrators to ensure optimal performance and maintain data integrity.

LIBRARIES

1. PYTHON LIBRARIES

WHAT IS A LIBRARY?

A library is a collection of precompiled codes that can be used later on in a program for some specific well-defined operations. Other than pre-compiled codes, a library may contain documentation, configuration data, message templates, classes, and values, etc.

A Python library is a collection of related modules. It contains bundles of code that can be used repeatedly in different programs. It makes Python Programming simpler and convenient for the programmer. As we don't need to write the same code again and again for different programs. Python libraries play a very vital role in fields of Machine Learning, Data Science, Data Visualization, etc.

HOW PYTHON LIBRARIES WORK?

A Python library is simply a collection of codes or modules of codes that we can use in a program for specific operations. We use libraries so that we don't need to write the code again in our program that is already available. But how it works. Actually, in the MS Windows environment, the library files have a DLL extension (Dynamic Load Libraries). When we link a library with our program and run that program, the linker automatically searches for that library. It extracts the functionalities of that library and interprets the program accordingly. That's how we use the methods of a library in our program.

TYPES OF LIBRARIES IN PYTHON?

1. The Python Standard Library

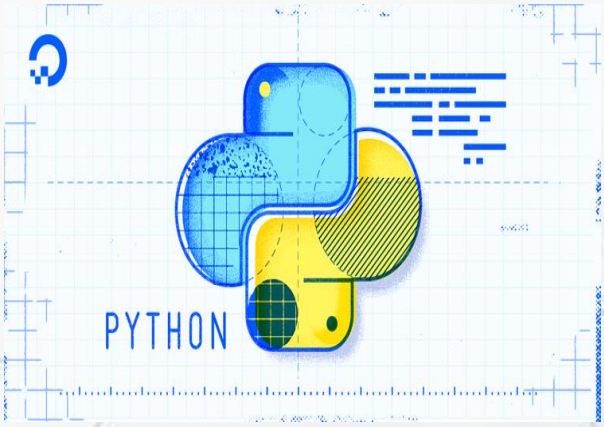
The Python Standard Library contains the exact syntax, semantics, and tokens of Python. It contains built-in modules that provide access to basic system functionality like I/O and some other core modules. Most of the Python Libraries are written in the C programming language. The Python standard library consists of more than 200 core modules. All these work together to make Python a high-level programming language.

Python Standard Library plays a very important role. Without it, the programmers can't have access to the functionalities of Python. But other than this, there are several other libraries in Python that make a programmer's life easier.

2. TensorFlow

This library was developed by Google in collaboration with the Brain Team. It is an open-source library used for high-level computations. It is also used in machine learning and deep learning algorithms.

3. Matplotlib



This library is responsible for plotting numerical data. And that's why it is used in data analysis. It is also an open-source library and plots high-defined figures like pie charts, histograms, scatterplots, graphs, etc.

4. Pandas

Pandas are an important library for data scientists. It is an open-source machine learning library that provides flexible high-level data structures and a variety of analysis tools. It eases data analysis, data manipulation, and cleaning of data.

Pandas support operations like Sorting, Re-indexing, Iteration, Concatenation, Conversion of data, Visualizations etc.

5. Numpy

The name “Numpy” stands for “Numerical Python”. It is the commonly used library. It is a popular machine learning library that supports large matrices and multi-dimensional data. It consists of in-built mathematical functions for easy computations.

6. SciPy

The name “SciPy” stands for “Scientific Python”. It is an open-source library used for high-level scientific computations. This library is built over an extension of Numpy. It works with Numpy to handle complex computations. It is also widely used by application developers and engineers.

7. Scikit-learn

It is a famous Python library to work with complex data. Scikit-learn is an open-source library that supports machine learning. It supports variously supervised and unsupervised algorithms.

8. PyGame

This library provides an easy interface to the Standard Directmedia Library (SDL) platform-independent graphics, audio, and input libraries. It is used for developing video games using computer graphics and audio libraries along with Python programming language.

9. PyTorch

PyTorch is the largest machine learning library that optimizes tensor computations. It has rich APIs to perform tensor computations with strong GPU acceleration.

10. PyBrain

The name "PyBrain" stands for Python Based Reinforcement Learning, Artificial Intelligence, and Neural Networks library. It is an open-source library built for beginners in the field of Machine Learning. It provides fast and easy-to-use algorithms for machine learning tasks. It is so flexible and easily understandable.

MODULE

- The act of partitioning a program into individual components (known as modules) is called MODULARITY.
- A module is a separate unit in itself. It can define functions, classes, and variables.

ADVANTAGES OF MODULE:

❖ Readability

Because of module programming, the code is straightforward to read. But it takes an experienced developer to conduct straightforward and clear modules.

❖ Easy testing and faster fixes

Module programming allows testing separate parts of the whole software. It saves time and ultimately makes it easier to perform the tests.

❖ Reusability

Its contents can be reused again and again in other programs, without having to rewrite or recreate them.

❖ Boundaries

It creates a number of well defined, documented boundaries within the program.

PYTHON MODULE

- A python module is a file containing Python definitions and statements. The file name is the module name with the suffix .py appended. Within a module, the module's name (as a string) is available as the value of the global variable __name__.
- We can import the functions, and classes defined in a module to another module using the import statement in some other Python source file.
- Grouping related code into a module makes the code easier to understand and use.
- Some of the modules from python's standard library are math, random, cmath, etc.

SOURCE CODE

With Breakdowns and working algorithm

1. Prerequisite imports:

```python
import mysql.connector
from mysql.connector import Error  # For better error handling
from prettytable import PrettyTable  # Importing PrettyTable for table display
import datetime

```

Uses:

- Database management (mysql.connector)
- Error handling (Error)
- Data presentation (PrettyTable)
- Time tracking (datetime)

2. Database Connection and Setup:

2.1)

```python
# Initial connection setup without specifying a database
def get_connection_initial():
    try:
        return mysql.connector.connect(
            host="localhost",
            user="root",
            password="Anubhavarya14022007"
        )
    except Error as e:
        print(f"Error connecting to MySQL: {e}")
        exit(1)

```

This function connects to MySQL without specifying a database. It handles errors and exits if the connection fails.

Steps:

1. Try to connect to MySQL server using root user and provided password.
2. If connection fails, print error and exit.
3. If successful, return the connection object.

2.2)

```python
# Connection setup with a specific database
def get_connection():
    try:
        return mysql.connector.connect(
            host="localhost",
            user="root",
            password="Anubhavarya14022007",
            database="ComputerSalesDB"
        )
    except Error as e:
        print(f"Error connecting to MySQL Database: {e}")
        exit(1)

```

This function connects to MySQL with a specific database.

Steps:

1. Try to connect to MySQL server using root user, provided password, and ComputerSalesDB database.
2. If connection fails, print error and exit.
3. If successful, return the connection object.

2.3)

```python
# Function to create the database if it doesn't exist
def create_database():
    connection = get_connection_initial()
    cursor = connection.cursor()
    cursor.execute("CREATE DATABASE IF NOT EXISTS ComputerSalesDB")
    print("Database 'ComputerSalesDB' is ready.")
    cursor.close()
    connection.close()

```

This function creates the database if it doesn't already exist.

Steps:

1. Establish an initial connection to MySQL without specifying a database.
2. Create a cursor to interact with the database.
3. Execute the SQL command to create ComputerSalesDB if it doesn't exist.
4. Print a confirmation message.
5. Close the cursor and connection.

2.4)

```python

# Function to create the table if it doesn't exist
def create_table():
    connection = get_connection()
    cursor = connection.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS computers (
            id INT AUTO_INCREMENT PRIMARY KEY,
            brand VARCHAR(50),
            model VARCHAR(50),
            price DECIMAL(10, 2),
            stock INT
        )
    """)
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS transactions (
            id INT AUTO_INCREMENT PRIMARY KEY,
            computer_id INT,
            quantity INT,
            total_price DECIMAL(10, 2),
            date_time DATETIME,
            customer_name VARCHAR(100),
            customer_address VARCHAR(255),
            customer_email VARCHAR(100)
        )
    """)
    print("Tables are ready.")
    cursor.close()
    connection.close()
```

This function creates two tables, computers and transactions, if they don't already exist in the ComputerSalesDB database:

- **computers table:** Stores computer details like ID, brand, model, price, and stock.
- **transactions table:** Tracks sales transactions, including computer ID, quantity sold, total price, date/time, and customer details.

It uses a database-specific connection, executes the SQL commands, and closes the connection.

3. Database Operations for Computers:

3.1)

```python
# Function to renumber IDs and reset AUTO_INCREMENT
def renumber_ids():
    connection = get_connection()
    cursor = connection.cursor()
    cursor.execute("SET @new_id = 0")
    cursor.execute("UPDATE computers SET id = (@new_id := @new_id + 1) ORDER BY id")
    connection.commit()
    print("IDs have been renumbered.")
    cursor.close()
    connection.close()
```

This function renumbers the id column in the computers table sequentially, ensuring consistent ordering:

- **Purpose:** It resets IDs to remove gaps after deletions, starting from 1.
- **Steps:**
    1. Initializes a variable @new_id to 0.
    2. Updates each id in ascending order, incrementing @new_id for each row.
    3. Commits the changes and closes the connection.

This maintains a clean and orderly ID sequence.

3.2)

```python

# Function to renumber transaction IDs and reset AUTO_INCREMENT
def renumber_transaction_ids():
    connection = get_connection()
    cursor = connection.cursor()

    try:
        #1: Renumber transaction IDs
        cursor.execute("SET @new_id = 0")
        cursor.execute("UPDATE transactions SET id = (@new_id := @new_id + 1) ORDER BY id")

        #2: Reset AUTO_INCREMENT to the next available ID (renumbering continuity)
        cursor.execute("ALTER TABLE transactions AUTO_INCREMENT = 1")
        connection.commit()

        print("Transaction IDs have been renumbered.")

    except Exception as e:
        print(f"Error renumbering transaction IDs: {e}")
    finally:
        cursor.close()
        connection.close()

```

This function renumbers the transaction IDs and resets the AUTO_INCREMENT value.

Steps:

1. Establish a connection to the database and create a cursor.
2. **Renumber IDs**: Set a variable @new_id to 0 and update transaction IDs in sequential order.
3. **Reset AUTO_INCREMENT**: Adjust the AUTO_INCREMENT value to ensure it continues from the next available ID.
4. Commit the changes and print a confirmation message.
5. Close the cursor and connection, ensuring resources are released.

3.3)

```python

# Add a new computer record
def add_computer(brand, model, price, stock):
    connection = get_connection()
    cursor = connection.cursor()
    query = "INSERT INTO computers (brand, model, price, stock) VALUES (%s, %s, %s, %s)"
    values = (brand, model, price, stock)
    cursor.execute(query, values)
    connection.commit()
    print("Computer added successfully.")
    cursor.close()
    connection.close()
    renumber_ids()

```

This function adds a new computer record to the database.

Steps:

1. **Get Connection**: Establish a connection to the database.
2. **Prepare Query**: Insert a new computer with the given brand, model, price, and stock.
3. **Execute Query**: Insert the values into the computers table.
4. **Commit**: Save the changes to the database.
5. **Close Connection**: Close the cursor and connection to free resources.
6. **Renumber IDs**: Call renumber_ids() to ensure that the computer IDs remain in a continuous sequence.

3.4)

```python

# View all computer records in a table format
def view_computers():
    connection = get_connection()
    cursor = connection.cursor()
    cursor.execute("SELECT * FROM computers")
    results = cursor.fetchall()
    table = PrettyTable()
    table.field_names = ["ID", "Brand", "Model", "Price", "Stock"]

    # Format price with the $ symbol for better presentation
    for row in results:
        row_with_price = list(row)
        row_with_price[3] = f"${row_with_price[3]:,.2f}"
        table.add_row(row_with_price)

    print(table)
    cursor.close()
    connection.close()


```

This function retrieves and displays all computer records in a formatted table.

Steps:

1. **Get Connection**: Establish a connection to the database.
2. **Query Data**: Execute a query to fetch all records from the computers table.
3. **Fetch Results**: Retrieve all rows returned by the query.
4. **Prepare Table**: Create a PrettyTable to display the data with headers: "ID", "Brand", "Model", "Price", and "Stock".
5. **Format Price**: Format the price column to include the dollar symbol and proper formatting.
6. **Add Rows**: Add each computer record to the table.
7. **Display Table**: Print the formatted table.
8. **Close Connection**: Close the cursor and connection to free resources.

3.5)

```python

# Update a computer record
def update_computer(id, price, stock):
    connection = get_connection()
    cursor = connection.cursor()
    query = "UPDATE computers SET price = %s, stock = %s WHERE id = %s"
    values = (price, stock, id)
    cursor.execute(query, values)
    connection.commit()
    print("Computer updated successfully.")
    cursor.close()
    connection.close()

```

This function updates the price and stock of a specific computer record based on the provided id.

Steps:

1. **Get Connection**: Establish a connection to the database.

2. **Prepare Query**: Write an SQL query to update the price and stock of a computer with a given id.
3. **Execute Query**: Pass the new values (price, stock, and id) and execute the update query.
4. **Commit Changes**: Commit the transaction to save the changes to the database.
5. **Close Connection**: Close the cursor and connection to free resources.
6. **Confirm Update**: Print a success message.

3.6)

```python

# Delete a computer record
def delete_computer(id):
    connection = None
    try:
        connection = get_connection()
        cursor = connection.cursor()

        #1: Update all transactions with this computer's ID to NULL to maintain referential integrity
        cursor.execute("UPDATE transactions SET computer_id = NULL WHERE computer_id = %s", (id,))
        connection.commit()

        #2: Delete the computer from the computers table
        cursor.execute("DELETE FROM computers WHERE id = %s", (id,))
        connection.commit()

        #3: Renumber IDs in the computers table
        renumber_ids()

        #4: Renumber transaction IDs to maintain consistency
        renumber_transaction_ids()

        print(f"Computer with ID {id} deleted and IDs renumbered successfully.")

    except Exception as e:
        print(f"Error deleting computer: {e}")
    finally:
        if connection:
            connection.close()

```

This function deletes a computer record by performing these steps:

1. **Update transactions**: Sets the computer ID to NULL in the transactions table to maintain integrity.
2. **Delete computer**: Removes the computer from the computers table.

3. **Renumber computer IDs**: Calls renumber_ids() to reassign IDs in the computers table.
4. **Renumber transaction IDs**: Calls renumber_transaction_ids() to maintain ID sequence in transactions.

Finally, the database connection is closed, and a success message is shown.

4. Transaction Management:

4.1)

```python

# Checkout process
def checkout():
    connection = get_connection()
    cursor = connection.cursor()
    view_computers()
    try:
        computer_id = int(input("Enter the ID of the computer you want to purchase: "))
        quantity = int(input("Enter quantity: "))
        cursor.execute("SELECT price, stock FROM computers WHERE id = %s", (computer_id,))
        result = cursor.fetchone()
        if not result:
            print("Invalid computer ID.")
            return
        price, stock = result
        if quantity > stock:
            print("Insufficient stock available.")
            return
        total_price = price * quantity
        confirmation = input(f"Total price is ₹{total_price:.2f}. Confirm purchase? (yes/no): ").strip().lower()
        if confirmation != "yes":
            print("Purchase cancelled.")
            return
        customer_name = input("Name: ").strip()
        customer_address = input("Address: ").strip()
        customer_email = input("Email (optional): ").strip()
        if not customer_name or not customer_address:
            print("Name and address cannot be empty.")
            return
        cursor.execute("UPDATE computers SET stock = stock - %s WHERE id = %s", (quantity, computer_id))
        cursor.execute("""
            INSERT INTO transactions (computer_id, quantity, total_price, date_time, customer_name, customer_address, customer_email)
            VALUES (%s, %s, %s, %s, %s, %s, %s)
        """, (computer_id, quantity, total_price, datetime.datetime.now(), customer_name, customer_address, customer_email))
        connection.commit()
        print("Purchase successful! Generating invoice...")
        generate_invoice(computer_id, quantity, total_price, customer_name, customer_address, customer_email)
    except ValueError:
        print("Invalid input. Please enter numeric values for IDs and quantities.")
    finally:
        cursor.close()
        connection.close()
```

This function facilitates the checkout process in several steps:

1. **View Computers**: Displays available computers for the customer.
2. **Input Details**: Takes the computer_id and desired quantity.
3. **Validation**:
    - Checks if the computer_id exists.
    - Ensures stock is sufficient.
4. **Calculate Total Price**: Computes the cost based on the quantity.
5. **Customer Details**: Collects mandatory details like name and address.
6. **Confirm Purchase**: Asks the user to confirm before proceeding.
7. **Update Stock**: Reduces stock in the computers table.
8. **Record Transaction**: Logs the purchase in the transactions table.
9. **Generate Invoice**: Calls generate_invoice() for receipt creation.

Handles invalid inputs and exceptions gracefully, ensuring a smooth process.

4.2)

```python
# Generate and print the invoice
def generate_invoice(computer_id, quantity, total_price, customer_name, customer_address, customer_email):
    connection = get_connection()
    cursor = connection.cursor()
    cursor.execute("SELECT brand, model FROM computers WHERE id = %s", (computer_id,))
    computer = cursor.fetchone()
    if not computer:
        print("Invalid computer ID.")
        return
    brand, model = computer
    date_time = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    table = PrettyTable()
    table.field_names = ["Invoice Details", "Information"]
    table.add_row(["Invoice Date", date_time])
    table.add_row(["Customer Name", customer_name])
    table.add_row(["Customer Address", customer_address])
    table.add_row(["Customer Email", customer_email or "N/A"])
    table.add_row(["Brand", brand])
    table.add_row(["Model", model])
    table.add_row(["Quantity", quantity])
    table.add_row(["Total Price", f"₹{total_price:.2f}"])
    print("\n" + "=" * 40)
    print(f"INVOICE - Purchase Date: {date_time}")
    print(table)
    print("=" * 40)
    print("Thank you for your purchase!")
    print("=" * 40)
    cursor.close()
    connection.close()
```

This function generates and displays a detailed invoice for a completed transaction:

1. **Fetch Computer Details**: Retrieves the brand and model using computer_id.
2. **Validate Computer**: Ensures the computer_id is valid before proceeding.
3. **Generate Invoice Table**:
    - Adds essential transaction details (e.g., date, customer info, computer details).
    - Formats price and includes optional fields like email.
4. **Print Invoice**:
    - Displays invoice in a clear and professional table format.
    - Adds separators for better readability.

5. **Close Connection**: Cleans up database resources after use.

A visually organized invoice ensures transparency and customer satisfaction.

4.3)

```python
# To delete transactions
def delete_transaction(transaction_id):
    try:
        connection = get_connection()
        cursor = connection.cursor()

        # Deleting the transaction based on the given transaction ID
        query = "DELETE FROM transactions WHERE id = %s"
        cursor.execute(query, (transaction_id,))
        connection.commit()

        print(f"Transaction with ID {transaction_id} deleted successfully.")

        # Renumber transaction IDs after deletion
        renumber_transaction_ids()

        cursor.close()
        connection.close()
    except Exception as e:
        print(f"Error deleting transaction: {e}")
        if connection:
            connection.close()
```

This function deletes a specific transaction record from the database:

1. **Delete Transaction**: Executes a DELETE query to remove the transaction using transaction_id.
2. **Commit Changes**: Saves the deletion in the database.
3. **Renumber IDs**: Calls renumber_transaction_ids() to maintain ID sequence.
4. **Error Handling**: Catches and logs errors during deletion.
5. **Resource Management**: Closes database resources after completion.

Efficient for removing specific transactions while keeping database integrity.

5. Analytics and Alerts:

5.1)

```python
# Get current statistics
def get_statistics():
    connection = get_connection()
    cursor = connection.cursor()
    cursor.execute("SELECT COUNT(*) FROM computers")
    total_computers = cursor.fetchone()[0]
    cursor.execute("SELECT SUM(price * stock) FROM computers")
    total_value = cursor.fetchone()[0] or 0
    cursor.execute("SELECT AVG(price) FROM computers")
    average_price = cursor.fetchone()[0] or 0

    # Create PrettyTable to display the statistics
    table = PrettyTable()
    table.field_names = ["Statistic", "Value"]

    # Format total_value and average_price with $ symbol and commas
    formatted_total_value = f"${total_value:,.2f}"
    formatted_average_price = f"${average_price:,.2f}"

    table.add_row(["Total Computers", total_computers])
    table.add_row(["Total Stock Value", formatted_total_value])
    table.add_row(["Average Price", formatted_average_price])


    print(table)

    cursor.close()
    connection.close()
```

This function retrieves and displays inventory statistics:

1. **Total Computers**: Fetches the count of all records in the computers table.
2. **Total Stock Value**: Calculates the sum of all price × stock.
3. **Average Price**: Computes the average price of all computers.
4. **PrettyTable Formatting**: Presents results in a structured table with clear labels.
5. **Resource Handling**: Closes database connections after execution.

Useful for monitoring inventory and financial summaries.

5.2)

```python
# Low Stock alert
def stock_alert(threshold=5):
    connection = get_connection()
    cursor = connection.cursor()
    cursor.execute("SELECT id, brand, model, stock FROM computers WHERE stock < %s", (threshold,))
    rows = cursor.fetchall()
    if rows:
        print("Low stock alert:")
        table = PrettyTable()
        table.field_names = ["ID", "Brand", "Model", "Stock"]
        for row in rows:
            table.add_row(row)
        print(table)
    else:
        print("All items are well-stocked.")

    cursor.close()
    connection.close()
```

This function raises a **low-stock alert** for inventory:

1. **Threshold Check**: Fetches all computers with stock below the specified threshold (default: 5).
2. **Structured Output**: Uses PrettyTable to display results in a neat tabular format with ID, Brand, Model, and Stock details.
3. **Notification**: Prints "Low stock alert" with the table or confirms sufficient stock if no items are below the threshold.
4. **Resource Handling**: Ensures database connections are properly closed post-execution.

Helps in timely inventory restocking.

6. Reports:

```python
# Sales Report
def sales_report():
    try:
        print("Generating sales report...")

        # Renumber transaction IDs before generating the report
        renumber_transaction_ids()

        connection = get_connection()
        cursor = connection.cursor()

        # Fetch transactions with related computer details and customer info
        cursor.execute("""
            SELECT
                t.id, c.brand, c.model, t.quantity, t.total_price, t.date_time, t.customer_name,
                t.customer_address, t.customer_email
            FROM transactions t
            INNER JOIN computers c ON t.computer_id = c.id
        """)
        rows = cursor.fetchall()

        # Print the report
        table = PrettyTable()
        table.field_names = ["Transaction ID",
                             "Brand",
                             "Model",
                             "Quantity",
                             "Total Price",
                             "Date & Time",
                             "Customer Name",
                             "Customer Address",
                             "Customer Email"]
        for row in rows:
            # Format the total_price with $ symbol and proper number formatting
            formatted_price = f"${row[4]:,.2f}"
            row_with_price = list(row)
            row_with_price[4] = formatted_price

            table.add_row(row_with_price)

        print(table)

        cursor.close()
        connection.close()

        print("Sales report generated. Returning to main menu...")
        return

    except Exception as e:
        print(f"Error generating sales report: {e}")
        if connection:
            connection.close()
        return
```

This function generates a **detailed sales report**:

1. **ID Renumbering**: Ensures transaction IDs are sequential by calling renumber_transaction_ids().

2. **Data Retrieval**: Fetches transactions with computer details (brand, model) and customer information (name, address, email) using a SQL query with a join.
3. **Report Formatting**: Displays data in a tabular format using PrettyTable. The total price is formatted for better readability (e.g., $1,000.00).
4. **Comprehensive View**: Includes transaction ID, computer details, quantity, total price, date & time, and customer details.
5. **Resource Management**: Closes the database connection post-execution to ensure no leaks.
6. **Error Handling**: Prints appropriate error messages if issues occur during execution.

Supports insightful review of sales performance.

7. Main Menu:

7.1)

```python
# Main menu
def main():
    create_database()
    create_table()
    while True:

        print("\n" + "Computer Sales Management System".center(50, "-"))
        print("1. Add Computer")
        print("2. View Computers")
        print("3. Update Computer")
        print("4. Delete Computer")
        print("5. View Current Statistics")
        print("6. Checkout")
        print("7. Sales Report")
        print("8. Delete Transaction")
        print("9. Stock Alert")
        print("10. Exit")
```

Functions:

1. Add Computer (choice = 1)

2. View Computers (choice = 2)
3. Update Computer (choice = 3)
4. Delete Computer (choice = 4)
5. View Current Statistics (choice = 5)
6. Checkout (choice = 6)
7. Sales Report (choice = 7)
8. Delete Transaction (choice = 8)
9. Stock Alert (choice = 9)
10. Exit (choice = 10)

7.2)

```python
        choice = input("Enter your choice: ").strip()
        if choice == "1":
            brand = input("Enter brand: ")
            model = input("Enter model: ")
            try:
                price = float(input("Enter price: "))
                stock = int(input("Enter stock: "))
                add_computer(brand, model, price, stock)
            except ValueError:
                print("Invalid input! Price must be a number, and stock must be an integer.")
        elif choice == "2":
            view_computers()
        elif choice == "3":
            try:
                id = int(input("Enter computer ID to update: "))
                price = float(input("Enter new price: "))
                stock = int(input("Enter new stock: "))
                update_computer(id, price, stock)
            except ValueError:
                print("Invalid input! Please enter valid numbers.")
        elif choice == "4":
            try:
                id = int(input("Enter computer ID to delete: "))
                delete_computer(id)
            except ValueError:
                print("Invalid input! ID must be an integer.")
        elif choice == "5":
            get_statistics()
        elif choice == "6":
            checkout()
        elif choice == "7":
            sales_report()
        elif choice == "8":
            try:
                transaction_id = int(input("Enter transaction ID to delete: "))  # Get transaction ID to delete
                delete_transaction(transaction_id)
            except ValueError:
                print("Invalid input! Transaction ID must be an integer.")
        elif choice == "9":
            stock_alert()
        elif choice == "10":
            print("Exiting program. Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")
```

This function serves as the main menu of the Computer Sales Management System, providing the user with a list of options to manage computers, transactions, and inventory.

Steps:

1. **Create Database & Tables**: The program starts by creating the database and tables if they don't exist.
2. **Display Menu**: Print a list of options, such as adding, viewing, updating, or deleting computers, viewing statistics, and generating reports.
3. **User Input**: Prompt the user to enter their choice and validate the input.
4. **Conditional Logic**:
   - Based on the user's input, the program executes the corresponding function (e.g., add a computer, view all computers, checkout, etc.).
   - If the user inputs invalid data (like non-numeric values where numbers are expected), an error message is displayed.
5. **Exit Condition**: The program continues to run in a loop until the user chooses to exit (option 10).
6. **Handle Errors**: Ensure that invalid inputs are handled gracefully with appropriate error messages.

This approach integrates all system functionality, providing a seamless user experience.

8. Program Entry Point:

```
if __name__ == "__main__":
    main()
```

Working:

1. **Check if script is run directly**:
   - if __name__ == "__main__": ensures the following code runs only when the script is executed directly.
2. **Call main()**:
   - The main() function is called to start the program, which initializes the database and runs an interactive menu for user input.

OUTPUT

(Functioning)

1. Database and Table Creation (Startup Actions):

1.1) Database and Tables ready:

```
Database 'ComputerSalesDB' is ready.
Tables are ready.
```

1.2) Table 1: Computers

Description:

	Field	Type	Null	Key	Default	Extra
▶	id	int	NO	PRI	NULL	auto_increment
	brand	varchar(50)	YES		NULL	
	model	varchar(50)	YES		NULL	
	price	decimal(10,2)	YES		NULL	
	stock	int	YES		NULL	

Table Contents:

id	brand	model	price	stock
1	Apple	MacBook Pro 16	249999.00	5
2	Dell	XPS 15	199999.00	7
3	Razer	Blade 15 Advanced	259999.00	4
4	Lenovo	ThinkPad X1 Carbon Gen 9	199999.00	6
5	Asus	ROG Zephyrus G15	199999.00	4
6	HP	Spectre x360 14	179999.00	8
7	Microsoft	Surface Laptop Studio	179999.00	3
8	Apple	MacBook Air M2	119999.00	8
9	Samsung	Galaxy Book Pro 360	299999.00	10
10	Acer	Predator Helios 300	139999.00	4
11	LG	Gram 17	129999.00	3
12	Asus	M54	299999.00	6
13	Apple	A154	299999.00	8
NULL	NULL	NULL	NULL	NULL

1.2) Table 2: Transactions

Description:

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
computer_id	int	YES	MUL	NULL	
quantity	int	YES		NULL	
total_price	decimal(10,2)	YES		NULL	
date_time	datetime	YES		NULL	
customer_name	varchar(100)	YES		NULL	
customer_address	varchar(200)	YES		NULL	
customer_email	varchar(100)	YES		NULL	
brand	varchar(50)	YES		NULL	
model	varchar(50)	YES		NULL	
price	decimal(10,2)	YES		NULL	

Table Contents:

id	computer_id	quantity	total_price	date_time	customer_name	customer_address	customer_email	brand	model	price
1	1	1	249999.00	2024-11-28 01:00:06	Anubhav Arya	10, Ajay Nagar, FBD, HR	anubhav@yahoo.co.in	NULL	NULL	NULL
2	6	1	179999.00	2024-11-28 01:01:03	Shivangi	12, Jane Street, NY, USA	shivangi@outlook.com	NULL	NULL	NULL
3	9	1	299999.00	2024-11-28 01:02:00	Vinay	19, Clock Tower, Spectre City, Uganda	vinay@gmail.com	NULL	NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

2. Main Menu:

```
---------Computer Sales Management System---------
1. Add Computer
2. View Computers
3. Update Computer
4. Delete Computer
5. View Current Statistics
6. Checkout
7. Sales Report
8. Delete Transaction
9. Stock Alert
10. Exit
Enter your choice:
```

3. Adding Records (Choice = 1):

```
Enter your choice: 1
Enter brand: Microsoft
Enter model: Surface Laptop 7th Edition AI
Enter price: 399999
Enter stock: 5
Computer added successfully.
IDs have been renumbered.
```

4. Viewing Records (Choice = 2):

```
Enter your choice: 2
+----+-----------+-----------------------------+------------+-------+
| ID |   Brand   |            Model            |    Price   | Stock |
+----+-----------+-----------------------------+------------+-------+
| 1  |   Apple   |        MacBook Pro 16       | $249,999.00 |   4   |
| 2  |   Dell    |            XPS 15           | $199,999.00 |   7   |
| 3  |   Razer   |      Blade 15 Advanced      | $259,999.00 |   4   |
| 4  |   Lenovo  |   ThinkPad X1 Carbon Gen 9  | $199,999.00 |   6   |
| 5  |   Asus    |       ROG Zephyrus G15      | $199,999.00 |   4   |
| 6  |   HP      |       Spectre x360 14       | $179,999.00 |   7   |
| 7  | Microsoft |    Surface Laptop Studio    | $179,999.00 |   3   |
| 8  |   Apple   |        MacBook Air M2       | $119,999.00 |   8   |
| 9  | Samsung   |     Galaxy Book Pro 360     | $299,999.00 |   9   |
| 10 |   Acer    |     Predator Helios 300     | $139,999.00 |   4   |
| 11 |   LG      |            Gram 17          | $129,999.00 |   3   |
| 12 |   Asus    |            M54              | $299,999.00 |   6   |
| 13 |   Apple   |            A154             | $299,999.00 |   8   |
| 14 | Microsoft | Surface Laptop 7th Edition AI | $399,999.00 |   5   |
+----+-----------+-----------------------------+------------+-------+
```

5. Updating Records (Price and Stock) (Choice = 3):

```
Enter your choice: 3
Enter computer ID to update: 13
Enter new price: 499999
Enter new stock: 6
Computer updated successfully.
```

ID	Brand	Model	Price	Stock
1	Apple	MacBook Pro 16	$249,999.00	4
2	Dell	XPS 15	$199,999.00	7
3	Razer	Blade 15 Advanced	$259,999.00	4
4	Lenovo	ThinkPad X1 Carbon Gen 9	$199,999.00	6
5	Asus	ROG Zephyrus G15	$199,999.00	4
6	HP	Spectre x360 14	$179,999.00	7
7	Microsoft	Surface Laptop Studio	$179,999.00	3
8	Apple	MacBook Air M2	$119,999.00	8
9	Samsung	Galaxy Book Pro 360	$299,999.00	9
10	Acer	Predator Helios 300	$139,999.00	4
11	LG	Gram 17	$129,999.00	3
12	Asus	M54	$299,999.00	6
13	Apple	A154	$499,999.00	6
14	Microsoft	Surface Laptop 7th Edition AI	$399,999.00	5

6. Delete a Computer (Choice = 4):

```
Enter your choice: 4
Enter computer ID to delete: 12
IDs have been renumbered.
Transaction IDs have been renumbered.
Computer with ID 12 deleted and IDs renumbered successfully.
```

ID	Brand	Model	Price	Stock
1	Apple	MacBook Pro 16	$249,999.00	4
2	Dell	XPS 15	$199,999.00	7
3	Razer	Blade 15 Advanced	$259,999.00	4
4	Lenovo	ThinkPad X1 Carbon Gen 9	$199,999.00	6
5	Asus	ROG Zephyrus G15	$199,999.00	4
6	HP	Spectre x360 14	$179,999.00	7
7	Microsoft	Surface Laptop Studio	$179,999.00	3
8	Apple	MacBook Air M2	$119,999.00	8
9	Samsung	Galaxy Book Pro 360	$299,999.00	9
10	Acer	Predator Helios 300	$139,999.00	4
11	LG	Gram 17	$129,999.00	3
12	Apple	A154	$499,999.00	6
13	Microsoft	Surface Laptop 7th Edition AI	$399,999.00	5

7. View Current Statistics (Choice = 5):

```
Enter your choice: 5
+-------------------+----------------+
|     Statistic     |     Value      |
+-------------------+----------------+
|  Total Computers  |       13       |
| Total Stock Value | $16,849,930.00 |
|   Average Price   |  $235,383.62   |
+-------------------+----------------+
```

8. Checkout and Invoice Generation (Choice = 6):

8.1) Checkout:

```
Enter your choice: 6
+----+-----------+-------------------------------+-------------+-------+
| ID |   Brand   |             Model             |    Price    | Stock |
+----+-----------+-------------------------------+-------------+-------+
| 1  |   Apple   |         MacBook Pro 16        | $249,999.00 |   4   |
| 2  |    Dell   |             XPS 15            | $199,999.00 |   7   |
| 3  |   Razer   |       Blade 15 Advanced       | $259,999.00 |   4   |
| 4  |   Lenovo  |    ThinkPad X1 Carbon Gen 9   | $199,999.00 |   6   |
| 5  |    Asus   |        ROG Zephyrus G15       | $199,999.00 |   4   |
| 6  |     HP    |        Spectre x360 14        | $179,999.00 |   7   |
| 7  | Microsoft |     Surface Laptop Studio     | $179,999.00 |   3   |
| 8  |   Apple   |         MacBook Air M2        | $119,999.00 |   8   |
| 9  |  Samsung  |      Galaxy Book Pro 360      | $299,999.00 |   9   |
| 10 |    Acer   |      Predator Helios 300      | $139,999.00 |   4   |
| 11 |     LG    |             Gram 17           | $129,999.00 |   3   |
| 12 |   Apple   |              A154             | $499,999.00 |   6   |
| 13 | Microsoft | Surface Laptop 7th Edition AI | $399,999.00 |   5   |
+----+-----------+-------------------------------+-------------+-------+
Enter the ID of the computer you want to purchase: 13
Enter quantity: 2
Total price is ₹799998.00. Confirm purchase? (yes/no): Yes
Name: Poonam
Address: C-237, Hari Nagar, Bhubaneshwar, Odisha
Email (optional): poonam@hotmail.com
Purchase successful! Generating invoice...
```

8.2) Invoice Generation:

```
========================================
INVOICE - Purchase Date: 2024-11-28 01:43:50
+-------------------+-----------------------------------------+
| Invoice Details   |                Information              |
+-------------------+-----------------------------------------+
|   Invoice Date    |           2024-11-28 01:43:50           |
|  Customer Name    |                  Poonam                 |
| Customer Address  | C-237, Hari Nagar, Bhubaneshwar, Odisha |
|  Customer Email   |            poonam@hotmail.com           |
|      Brand        |                Microsoft                |
|      Model        |      Surface Laptop 7th Edition AI      |
|     Quantity      |                    2                    |
|   Total Price     |                ₹799998.00               |
+-------------------+-----------------------------------------+
========================================
Thank you for your purchase!
========================================
```

9.Printing Sales Report (Choice=7):

```
Enter your choice: 7
Generating sales report...
Transaction IDs have been renumbered.
+---------------+----------+----------------------------+----------+-------------+---------------------+---------------+-----------------------------------+---------------------+
| Transaction ID |  Brand   |           Model            | Quantity | Total Price |     Date & Time     | Customer Name |          Customer Address         |    Customer Email   |
+---------------+----------+----------------------------+----------+-------------+---------------------+---------------+-----------------------------------+---------------------+
|       1       |  Apple   |        MacBook Pro 16      |    1     | $249,999.00 | 2024-11-28 01:00:06 | Anubhav Arya  |       10, Ajay Nagar, FBD, HR     | anubhav@yahoo.co.in |
|       2       |    HP    |       Spectre x360 14      |    1     | $179,999.00 | 2024-11-28 01:01:03 |   Shivangi    |       12, Jane Street, NY, USA    | shivangi@outlook.com |
|       3       | Samsung  |     Galaxy Book Pro 360    |    1     | $299,999.00 | 2024-11-28 01:02:00 |    Vinay      | 19, Clock Tower, Spectre City, Uganda |  vinay@gmail.com  |
|       4       | Microsoft | Surface Laptop 7th Edition AI |    2     | $799,998.00 | 2024-11-28 01:43:51 |    Poonam     | C-237, Hari Nagar, Bhubaneshwar, Odisha |  poonam@hotmail.com |
+---------------+----------+----------------------------+----------+-------------+---------------------+---------------+-----------------------------------+---------------------+
Sales report generated. Returning to main menu...
```

For clear visibility:

```
Enter your choice: 7
Generating sales report...
Transaction IDs have been renumbered.
+---------------+----------+----------------------------+----------+-------------+---------------------+
| Transaction ID |  Brand   |           Model            | Quantity | Total Price |     Date & Time     |
+---------------+----------+----------------------------+----------+-------------+---------------------+
|       1       |  Apple   |        MacBook Pro 16      |    1     | $249,999.00 | 2024-11-28 01:00:06 |
|       2       |    HP    |       Spectre x360 14      |    1     | $179,999.00 | 2024-11-28 01:01:03 |
|       3       | Samsung  |     Galaxy Book Pro 360    |    1     | $299,999.00 | 2024-11-28 01:02:00 |
|       4       | Microsoft | Surface Laptop 7th Edition AI |    2     | $799,998.00 | 2024-11-28 01:43:51 |
+---------------+----------+----------------------------+----------+-------------+---------------------+
Sales report generated. Returning to main menu...
```

```
+---------------+-----------------------------------------+---------------------+
| Customer Name |           Customer Address              |   Customer Email    |
+---------------+-----------------------------------------+---------------------+
| Anubhav Arya  |        10, Ajay Nagar, FBD, HR          | anubhav@yahoo.co.in |
|   Shivangi    |        12, Jane Street, NY, USA         | shivangi@outlook.com |
|    Vinay      | 19, Clock Tower, Spectre City, Uganda   |  vinay@gmail.com    |
|    Poonam     | C-237, Hari Nagar, Bhubaneshwar, Odisha |  poonam@hotmail.com |
+---------------+-----------------------------------------+---------------------+
```

10. Delete Transactions (Choice=8):

```
Enter your choice: 8
Enter transaction ID to delete: 3
Transaction with ID 3 deleted successfully.
Transaction IDs have been renumbered.
```

Transaction ID	Brand	Model	Quantity	Total Price	Date & Time	Customer Name	Customer Address	Customer Email
1	Apple	MacBook Pro 16	1	$249,999.00	2024-11-28 01:00:06	Anubhav Arya	10, Ajay Nagar, FBD, HR	anubhav@yahoo.co.in
2	HP	Spectre x360 14	1	$179,999.00	2024-11-28 01:01:03	Shivangi	12, Jane Street, NY, USA	shivangi@outlook.com
3	Microsoft	Surface Laptop 7th Edition AI	2	$799,998.00	2024-11-28 01:43:51	Poonam	C-237, Hari Nagar, Bhubaneshwar, Odisha	poonam@hotmail.com

For clear visibility:

Transaction ID	Brand	Model	Quantity	Total Price	Date & Time
1	Apple	MacBook Pro 16	1	$249,999.00	2024-11-28 01:00:06
2	HP	Spectre x360 14	1	$179,999.00	2024-11-28 01:01:03
3	Microsoft	Surface Laptop 7th Edition AI	2	$799,998.00	2024-11-28 01:43:51

Customer Name	Customer Address	Customer Email
Anubhav Arya	10, Ajay Nagar, FBD, HR	anubhav@yahoo.co.in
Shivangi	12, Jane Street, NY, USA	shivangi@outlook.com
Poonam	C-237, Hari Nagar, Bhubaneshwar, Odisha	poonam@hotmail.com

11. Stock Alert (Choice=9):

Provides a list of Computers whose stock is less than 5.

```
Enter your choice: 9
Low stock alert:
```

ID	Brand	Model	Stock
1	Apple	MacBook Pro 16	4
3	Razer	Blade 15 Advanced	4
5	Asus	ROG Zephyrus G15	4
7	Microsoft	Surface Laptop Studio	3
10	Acer	Predator Helios 300	4
11	LG	Gram 17	3
13	Microsoft	Surface Laptop 7th Edition AI	3

12. Exiting the program (Choice=10):

```
Enter your choice: 10
Exiting program. Goodbye!
```

FUNCTIONS USED IN PROJECT

1. Add Computer (choice = 1)
2. View Computers (choice = 2)
3. Update Computer (choice = 3)
4. Delete Computer (choice = 4)
5. View Current Statistics (choice = 5)
6. Checkout (choice = 6)
7. Sales Report (choice = 7)
8. Delete Transaction (choice = 8)
9. Stock Alert (choice = 9)
10. Exit (choice = 10)

BIBLIOGRAPHY

1. Books:

- Computer Science with Python (Class 11th and 12th) ~ By Sumita Arora
- Class 12 CBSE Computer Science textbook for foundational knowledge in Python and SQL.

2. Online Resources:

- The official Python documentation (https://docs.python.org) for syntax, library functions, and advanced programming concepts.
- MySQL official documentation (**https://dev.mysql.com/doc**) for database management and queries.

3. Libraries:

- Documentation for the PrettyTable library (https://pypi.org/project/PrettyTable/) to create well-structured tables in Python.
- Python's built-in MySQL Connector module (https://dev.mysql.com/doc/connector-python/en/) for connecting Python to MySQL databases.

4. Online Tutorials and Forums:

- Stack Overflow (https://stackoverflow.com) – For troubleshooting and problem-solving during development.

- W3Schools (https://www.w3schools.com/) – For quick reference to SQL and Python syntax.

5. OpenAI Assistance:

- Guidance and suggestions provided by AI tools like OpenAI's ChatGPT for refining code and developing algorithms.

These references reflect the resources used in understanding, developing, and enhancing the "Computer Sales Service Management" project.

Thank
You!